

ENTERPRISE ACCESS FOUNDATION

Phase 1.5 - RECONCILED FINAL TARGET ARCHITECTURE

Authorization / Access / Scope / Sharing / Field Security / Approval / Policy / Territory

Research + Repository Review Reconciliation

Foundation-first, not speculative-platform-first

1. Final Executive Decision

The final Phase 1.5 is not a replacement for the old security baseline. It is the old baseline plus a deliberately narrow Enterprise Access Control Plane foundation.

```
FINAL PHASE 1.5
=
Existing Security Baseline
+
Enterprise Access Control Plane Foundation
```

Final recommendation: REVISE the old Phase 1.5 plan.

The earlier Role -> Permission-only plan was too narrow, but its tenant-safety, persistence, testing, host-wiring, evidence, idempotency and concurrency work remains necessary. The revised plan adds stable ActionKey, PermissionSet, Principal/Actor contracts, relation-aware grants, query authorization and revision semantics.

2. What Changed After the Claude Review

Topic	Earlier report	Reconciled decision
Old baseline	Effectively replaced	Keep it and build the control plane on top
Query scope	Tenant scope could be enough initially	Implement None / All / OwnedBy now
Principal	Mostly user-centric	Freeze PrincipalRef + ActorContext now
Network	Insufficiently modeled	Not an ordinary RoleAssignment scope
Approval	Single Approval bounded context	Split domain invariant, Workflow/Rules, Human Task/Approval runtime
Sharing / Field Security	Separate logical domains	Access subdomains/evaluators; not separate services
Admin grant safety	Not explicit	Delegated administration boundary, not 'may grant only what I personally have'
System catalog	Open	Do not freeze copy-on-provision reconciler yet
Expression language	Potential CEL later	No custom DSL now; do not choose CEL/Cedar/DMN yet

3. Repository Reality to Preserve

- Access currently needs the same production-hardening discipline already used in CRM/MasterData.
- Phase 2 business commands must not begin with caller-supplied TenantId/PrincipalRef as authority.
- Access must be wired through the Host/composition root before it can become an enforcement dependency.
- Tenant-scoped Access data requires tenant-safe keys/relationships and negative cross-tenant tests.
- Runtime database access must use an unprivileged application role; a PostgreSQL superuser can bypass RLS.
- Access needs its own integration/architecture/security test project before Phase 2 consumes it.

The strategic advantage is timing: Access is not yet deeply consumed by other modules, so the model can be corrected before handlers, HTTP APIs, search, exports, extensions, approval flows and agents depend on the wrong semantics.

4. Non-Negotiable Separation of Concerns

TENANT ISOLATION != BUSINESS AUTHORIZATION

AUTHORIZATION != ORGANIZATION
AUTHORIZATION != TERRITORY
AUTHORIZATION != APPROVAL
AUTHORIZATION != AUTOMATION

Inside Access:
Functional Grants

Relationship / Sharing
Restriction Policies
Field Security
remain distinct evaluators/subdomains.

5. Security Baseline - IMPLEMENT NOW

Baseline capability	Required outcome
Access.Tests	Integration, architecture and negative security tests
Unprivileged runtime DB role	NOSUPERUSER / NOBYPASSRLS runtime connection
Access tenant RLS	Hard tenant defense-in-depth
Tenant-safe relationships	Composite tenant-safe foreign keys where persistence requires them
Tenant uniqueness	Tenant-scoped names/codes must not collide incorrectly
Concurrency baseline	row_version / optimistic concurrency where Access config mutates
Host registration	Access participates in the real runtime composition root
Grant/Revoke commands	Server-side checked configuration mutations
Idempotency	Safe replay semantics for Access configuration commands
Outbox	Authorization-config change events/facts
Evidence / audit	Who changed security configuration, why and from what source
Architecture tests	Prevent modules from bypassing Contracts or depending on Access.Persistence

6. Canonical Access Control Plane

```
Action Registry
|
PermissionSet
|
PermissionGrant / PermissionSetItem
|
Role
|
RoleAssignment
|
Principal

Authorize(Principal, Action, Resource, Context)
ResolveAccessScope(Principal, Action, ResourceType, Context)
```

Permission is treated semantically as the right to invoke a business ActionKey. The platform registry stores the action vocabulary; PermissionSet items/grants bind actions to authorization constraints.

7. Action Registry

- ActionKeys are stable business capabilities, not routes, buttons or component names.
- The owning business module owns the vocabulary; Access evaluates it.
- Examples: crm.opportunity.read, crm.opportunity.win, sales.quote.approve.
- Action keys are immutable identifiers once published. Rename means deprecate + introduce a new key, not silent semantic mutation.
- Source code/manifest should be authoritative; a database table may be a validated projection for querying/admin metadata.
- CI should fail when application code uses an unregistered ActionKey.

Field	Purpose
action_key	Stable text identifier / primary semantic key
owner_module	CRM, Sales, Inventory, Finance, etc.
resource_type	Opportunity, Quote, Invoice, etc.
risk_class	Optional platform metadata for high-risk handling
status	Active / Deprecated

8. PrincipalRef and ActorContext

RoleAssignment must not remain permanently hard-coded to account_id. The authorization contract should be principal-aware even if Phase 1.5 only persists User assignments.

```
PrincipalRef
- type
- id

Future principal types:
User
ServicePrincipal
Agent
Group
Integration / Automation

ActorContext
- authenticated_principal
- active_tenant
- acting_for?      # future on-behalf-of / impersonation chain
- authentication_context
- correlation_id
```

Tenant context comes from the trusted authenticated session/context, not from the request command body.

9. Role, PermissionSet and Grant Semantics

```
Principal
-> RoleAssignment
    -> Role
        -> PermissionSet
            -> PermissionGrant(ActionKey, RelationConstraint?)
```

Role is a security persona. PermissionSet is a reusable duty/capability bundle. A grant may constrain an action by a relationship such as Owner. The action itself is not changed by the relationship.

Example persona	Action	Relation constraint
Sales Representative	crm.opportunity.read	Owner
Sales Representative	crm.opportunity.update	Owner
Sales Manager	crm.opportunity.read	None / unrestricted within assignment scope
Sales Manager	crm.opportunity.update	None / unrestricted within assignment scope

10. Relation and Scope - Implement Only What Is Real

Phase 1.5 should implement real semantics needed by Phase 2, not reserve unsupported database states.

Concept	Phase 1.5	Meaning
None	IMPLEMENT	No rows visible
All	IMPLEMENT	All rows allowed inside current tenant boundary
OwnedBy	IMPLEMENT	Rows whose business ownership relation matches principal
Organization scope	DESIGN/FREEZE	Future Organization fact provider; no active DB scope until Organization exists
Team	DESIGN/FREEZE	Distinguish org team from record/team relationship
Territory	DESIGN/FREEZE	Future commercial/business context, not Access-owned hierarchy
Sharing	DESIGN/FREEZE	Additive record grant inside Access subdomain

11. Query Authorization - First-Class Requirement

Collection authorization is not a loop of remote `Authorize()` calls. The query path needs a stable contract from day one.

```
Authorize(request)
  -> decision for one resource

ResolveAccessScope(request)
  -> None
  | All
  | OwnedBy(principal)
  | future AnyOf(...)
  | future InOrgNodes(...)
  | future InTerritories(...)
  | future SharedWith(...)
```

Critical invariant:

```
Authorize(actor, action, row) == ALLOW
IFF
row is included by ResolveAccessScope(actor, action, resource_type)
```

This equivalence should be tested. Unknown future scope terms must fail closed; list queries must filter in SQL/read-model translation rather than fetching unauthorized rows and post-filtering them.

12. Record Ownership

Owner semantics now matter because the first real Phase 2 authorization requirement is expected to distinguish users who can operate only on their own records from managers who can operate on all records in their tenant scope.

The owning domain supplies the ownership fact. Access must not reach directly into private CRM/Sales tables. The PEP/domain adapter provides the `ResourceRef` and required security facts to the PDP.

The exact CRM owner fact (`created_by`, `assigned_principal`, `owner_principal`, or another explicit property) must be frozen before Phase 2 commands are implemented.

13. Cross-Tenant Network Access

Network is NOT an ordinary `RoleAssignment` scope.

Tenant remains the hard isolation boundary. Manufacturer/dealer or group-network visibility must not be modeled as a magic `Network` enum value that silently crosses tenant RLS.

Scenario	Recommended model
User genuinely operates inside dealer tenant	Explicit membership/external membership in that tenant, then normal Access evaluation
HQ needs aggregated dealer analytics	Authorized cross-tenant analytics/read projection, not raw OLTP bypass
A specific record must cross tenant boundary	Future explicit inter-tenant/federation grant contract with provenance and audit

The existing `Tenant/OrganizationUnit/Network`-style scope enum should therefore not define the long-term authorization model.

14. RLS vs Application Authorization

Layer	Owns
PostgreSQL RLS	Hard tenant isolation / defense in depth
Access PDP	Within-tenant functional/resource authorization
Domain policy	Lifecycle, state and business invariants
Workflow/Approval	Required human/process consent where policy says approval is needed

Do not encode ownership, organization, territory, sharing and ABAC as the primary business authorization truth in RLS. The same policy must govern APIs, queries, workers, exports, search, extensions and AI.

15. Sharing and Field Security

```
ACCESS
|- Functional Authorization
|- Relationship / Sharing
|- Restriction Policies
`- Field Security
```

Sharing and Field Security are logical Access subdomains/evaluators, not necessarily separate deployable services.

- Sharing cannot manufacture a functional capability the user does not have.
- Field Security narrows an already-authorized record view/mutation.
- Field controls must apply to backend reads/writes, exports, search/index projections, extensions and AI context.
- Business domains classify which fields are security-sensitive; Access owns the reusable evaluation mechanism.

16. Restriction / Forbid Semantics

Normal functional grants are additive. Restrictive policies are a separate narrowing layer. Do not mix general explicit DENY entries into ordinary role inheritance in Phase 1.5.

```
ALLOW =
  UnionOfApplicableGrants
  AND ScopeSatisfied
  AND RelationSatisfied
  AND NOT AnyApplicableForbid
```

Restriction semantics should be order-independent. Phase 1.5 does not expose tenant-authored forbid policies; only the semantic seam is frozen.

17. Approval - Corrected Ownership Model

The earlier single 'Approval bounded context' box was too coarse. The platform should separate business ownership, rule/path selection and durable human task lifecycle.

```
CRM / SALES DOMAIN
owns:
- Is approval required before this business transition?
- Domain invariant after approval
```

|
v

```
WORKFLOW / RULES
owns:
- Approval policy / definition
- Immutable version
- Conditions / thresholds
```

- Path / approver resolver specification

|
v

HUMAN TASK / APPROVAL RUNTIME

owns:

- Assignment / candidate pool
- Claim / delegate / substitute
- Approve / reject / request-info
- SLA / escalation
- Decision evidence

|
v

WORK INBOX

- projection and action surface, not business source of truth

Approval permission/capability and 'this person is the assigned approver for this instance' remain separate facts.

18. Workflow, Automation and Conditions

Do not build one generic rule engine for Access, Approval, Territory, Sharing and Automation. A shared constrained expression library may emerge later, but each domain keeps its own lifecycle and semantics.

Concern	Typical input	Output
Authorization	Principal + Action + Resource + Context	Permit / deny
Workflow decision	Business facts + policy version	Approval required/path/resolvers
Automation	Trigger + conditions	Authorized actions
Territory assignment	Business/resource facts	Territory assignment
Sharing rule	Record criteria	Additive record grant

Expression-language decision:

Do not create a custom DSL/AST now. Also do not freeze CEL, Cedar, DMN or another engine now. Freeze only a future evaluator/condition seam; choose technology when real rule complexity appears.

19. Audit, Evidence and TenantAccessRevision

ConfigurationRevision and 'decision epoch' should be one concept: TenantAccessRevision.

TenantAccessRevision

- monotonic per tenant
- increments when effective authorization configuration changes
- carried in authorization decisions
- can later support cache invalidation / revocation observability

Concern	Persistence/telemetry rule
Configuration audit	Durable before/after + actor + reason/source + revision
Assignment provenance	granted_by, source, reason, effective_from, effective_until
Normal read decisions	Structured telemetry/logging; not a synchronous row insert per read
High-risk mutation decisions	DecisionId referenced by durable domain/business evidence
Approval decision	Pinned to immutable workflow/approval definition version

20. Delegated Administration - Correct Safety Rule

Do NOT enforce the simplistic rule 'an admin may grant only permissions they personally possess'. That would prevent a Security Administrator from assigning a Finance role they are intentionally not allowed to exercise.

```
CanExercise(permission)
!=
CanGrant(permission / role)
```

The long-term model is a delegated administration boundary: an administrator may grant only roles/capabilities inside the administrative boundary explicitly delegated to them. Phase 1.5 must prevent unrestricted escalation, but the exact boundary model can remain narrow until an administration UI/workflow exists.

21. System Catalog Strategy - OPEN DECISION

Do not freeze a permanent copy-on-provision + continuous reconciler design yet. It is valid but creates drift, reconciliation and tenant customization semantics before the product needs them.

- Action Registry is platform-controlled.
- Roles and PermissionSets can initially be tenant-local.
- System role/permission-set templates may be added later if repeatable provisioning becomes a concrete requirement.
- Whether templates are copied once, reconciled continuously or referenced as immutable platform definitions remains an owner decision.

22. Revocation and Cache Semantics - FREEZE, IMPLEMENT LATER

The architecture must define that high-risk authorization revocation propagates quickly and fails closed, but Phase 1.5 does not need a distributed cache-invalidation platform if authorization remains local/in-process.

TenantAccessRevision is the seam that later allows cached authorization state to be invalidated or rejected when stale.

23. Idempotency and Authorization Ordering

An idempotency replay must not become an authorization bypass. A previously successful response cannot automatically be returned to a principal whose tenant/resource authorization is no longer valid.

```
Authenticate / resolve trusted tenant
-> minimum authorization context
-> idempotency lookup bound to tenant + principal/action/request fingerprint
-> authorization/business validation as required by replay policy
-> execute or safely replay
```

The exact replay policy should be codified once in the Phase 1.5/Phase 2 handler template rather than reimplemented per command.

24. Reconciled Authorization Evaluation Pipeline

1. Authenticate
2. Build trusted ActorContext / active tenant
3. Tenant membership + tenant state gate
4. (Future) entitlement gate
5. Validate ActionKey
6. PEP loads/provides ResourceRef + security facts
7. PDP evaluates:

- grants UNION
- AND assignment scope
- AND relation constraints
- AND NOT forbids
- 8. Return decision + obligations + TenantAccessRevision
- 9. Apply field security where applicable
- 10. Domain business policy
- 11. Workflow/approval requirement and current approval state
- 12. Domain invariants
- 13. Idempotent mutation / commit / evidence / outbox

For CREATE actions where no resource instance exists yet, ResourceRef may be a type-level resource descriptor plus target tenant/organization context supplied by the PEP. Access never reads private domain tables directly.

25. Phase 1.5 - IMPLEMENT NOW

1. Access.Tests + architecture/security integration tests
2. Unprivileged runtime DB role and RLS enforcement tests
3. Tenant-safe Access persistence constraints / composite FKs / uniqueness
4. Concurrency baseline (row_version where configuration mutates)
5. Host/composition-root registration
6. Contracts: ActorContext, PrincipalRef, ResourceRef, ActionKey, AuthorizationRequest/Decision
7. Action Registry + source/manifest consistency CI test
8. PermissionSet + PermissionGrant/PermissionSetItem
9. Role + Role -> PermissionSet
10. RoleAssignment with effective period and provenance
11. TenantAccessRevision
12. In-process default-deny PDP / Authorize()
13. Relation constraint: Owner
14. ResolveAccessScope(): None / All / OwnedBy
15. Authorize vs query-scope equivalence tests
16. Server-side PEP pattern / adapter contract
17. Grant/Revoke commands with idempotency + evidence + outbox
18. Decision telemetry and reason codes
19. Delegated-administration guard baseline (no unrestricted tenant-admin escalation)
20. Remove/retire legacy Role -> Permission and ambiguous Network-scope semantics as required by migration plan

26. DESIGN / FREEZE NOW, IMPLEMENT LATER

- Organization scopes and hierarchy resolver
- Team semantics: organization team vs record/team relationship
- Sharing evaluator and grant provenance
- Field Security evaluator/masking
- Restriction / forbid policies
- Territory context and versioned territory facts
- ServicePrincipal / Agent runtime assignments
- ActingFor / impersonation chain
- Direct PermissionSet assignment
- Delegation / JIT / break-glass / SoD / access review
- Step-up authentication obligations
- Entitlement gate
- Cache/revocation propagation implementation

- Cross-tenant network/federation grant model
- Approval resolver catalog and human-task runtime
- System role/permission-set template strategy

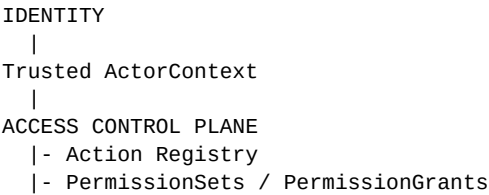
27. DO NOT BUILD YET

- Generic all-purpose Rules table/engine
- Custom expression DSL or custom generic AST
- OPA runtime dependency
- OpenFGA runtime dependency
- Cedar runtime dependency
- CEL commitment
- Territory engine/tables
- Organization authorization engine
- Sharing administration engine/UI
- Field Security administration engine/UI
- Approval runtime / workflow designer
- SoD/JIT/access-review platform
- Distributed policy microservice before a real deployment boundary exists

28. Freeze Decisions Before Implementation

Decision	Status
Tenant is hard isolation boundary	FREEZE
Runtime DB role is unprivileged and RLS cannot be bypassed	FREEZE
Default deny	FREEZE
ActionKey is stable domain-owned business vocabulary	FREEZE
Role -> PermissionSet -> PermissionGrant(ActionKey, constraint)	FREEZE
Role != organization hierarchy	FREEZE
Principal/Actor contracts are not hard-coded to account_id	FREEZE
Owner relation is implemented in Phase 1.5	FREEZE
Query scope supports None / All / OwnedBy now	FREEZE
Single-resource and query authorization must be equivalent	FREEZE
Organization/Territory are fact providers, not Access-owned hierarchies	FREEZE
Network is not a normal assignment scope	FREEZE
Sharing cannot exceed functional capability	FREEZE
Sharing + Field Security are Access subdomains/evaluators	FREEZE
Restrictions are separate, order-independent narrowing semantics	FREEZE
Approval ownership is split across Domain + Workflow/Rules + Human Task runtime	FREEZE
TenantAccessRevision is the single access-config epoch/revision concept	FREEZE
Configuration audit != decision telemetry != business evidence	FREEZE
CanExercise != CanGrant; delegated administration boundary	FREEZE
No custom DSL and no policy-engine vendor choice now	FREEZE
System catalog copy/reconcile strategy	OPEN
Exact CRM ownership fact for Opportunity	OWNER DECISION BEFORE PHASE 2

29. Final Target Architecture



```

|- Roles / Assignments
|- Owner relation
|- PDP / PEP
|- Query Scope
|- TenantAccessRevision
|- Sharing / Restriction / Field Security seams
|
+---- consumes Organization facts
+---- consumes Territory facts
|
AuthorizationDecision
|
DOMAIN (CRM / Sales / ...)
|- business state / lifecycle invariant
|- determines whether approval is required
|
WORKFLOW / RULES
|- versioned approval policy/path/resolver spec
|
HUMAN TASK / APPROVAL RUNTIME
|- assignment / decision / SLA / evidence
|
DOMAIN COMMAND / COMMIT / OUTBOX / EVIDENCE

Cross-tenant network:
separate membership / analytics / federation capability,
never a magic RoleAssignment scope.

```

30. Final Recommendation

FINAL VERDICT: REVISE the old Phase 1.5 plan.

The correct implementation boundary is neither the old Role -> Permission baseline nor a speculative enterprise IAM platform. Phase 1.5 should first harden Access to the same tenant-safe operational standard as CRM/MasterData, then add the smallest stable control-plane model that Phase 2 actually needs: trusted actor context, ActionKeys, PermissionSets, Roles, effective assignments, Owner relation, None/All/OwnedBy query scope, default-deny evaluation, revision/audit and enforcement contracts.

Everything else - organization hierarchy, territory, sharing, field security, ABAC, agents, cross-tenant federation, approvals and automation - receives a stable architectural seam now and a runtime only when a concrete phase requires it.

31. Benchmark and Reference Basis

- Salesforce - Permission Sets/Groups, Sharing Model, Role Hierarchy, Enterprise Territory Management; Restriction Rules limitations.
- Microsoft Dataverse - cumulative privileges, User/Business Unit/Parent-Child/Organization access depth, teams/sharing, column security.
- SAP - business roles/catalogs and organizational authorization restrictions; Flexible Workflow / Responsibility Management.
- Oracle Fusion - Job/Duty/Privilege, data security policies, approver types and versioned approval policy behavior.
- ServiceNow - ACL evaluation and Deny-Unless / Allow-If style restrictive controls.
- HubSpot / Zoho - own/team/all record visibility, sharing, permissions and approval patterns.
- NIST SP 800-162 - ABAC conceptual model.
- Cedar - Principal/Action/Resource/Context and permit/forbid semantics.
- OpenFGA - relationship-based authorization and immutable model concepts.
- OPA - authorization filtering/search and asynchronous decision-log concepts.

- Existing Enterprise Access, Organization, Workflow/Rules, Work Inbox, Search, Extension and Quality architecture research in the project.

Reconciled Final - Phase 1.5 Architecture Decision Report